

Assignment: Triple-Camera Live Review System

Overview

We are building a real-time multi-camera sports review system for analysts and referees. The system must support synchronized live playback, instant camera switching, event-based navigation, and replay workflows without additional network downloads during review mode.

The goal of this assignment is to evaluate:

- System architecture thinking
 - Frontend + backend integration capability
 - Real-time streaming understanding
 - Video synchronization handling
 - React/hls.js engineering quality
 - Clean code structure and execution speed
-

Existing Infrastructure

The following systems and datasets already exist and should be reused.

1. Camera Segment Streams

Three synchronized MPEG-TS segment streams already exist on disk:

SOURCE camera

`sync_reports/ts_segments_source/1645/`

SINK camera

`sync_reports/ts_segments_sink/1645/`

HQ camera

`sync_reports/ts_segments_hq/1645/`

Each contains:

- `.ts` segment files
 - `playlist.m3u8`
-

2. Segment ↔ Frame Mapping CSVs

These files map every segment to the exact frame ranges it contains.

Files

- `test_work/cv_output/reader/source/hls_segment_frame_index.csv`
- `test_work/cv_output/reader/sink/hls_segment_frame_index.csv`
- `test_work/cv_output/reader/hq/hls_segment_frame_index.csv`

Format:

`segment_index, seg_basename, cumulative_start_frame, frame_count`

These files are critical for converting:

- frame numbers
- sync indices
- bounce events

into actual playback positions.

3. Triple-Camera Sync Table

File

`sync_reports/segments_1645/sync/hls_sync_1645_triple.csv`

Contains:

- Source frame index
- Sink frame index

- HQ frame index
- Nanosecond wall-clock timestamps

This allows exact temporal mapping between all three streams.

4. Shot / Event Data

File

`test_work/cv_output/correlation/flight_shots.csv`

Contains:

- shot_id
- bounce frames
- event timing
- 3D bounce coordinates
- counts_as_shot flag

These events become:

- timeline markers
 - replay jump points
 - event panel metadata
-

5. Bounce Clips

Directories:

- `bounce_clips_share/source/`
- `bounce_clips_share/sink/`
- `bounce_clips_share/hq/`

Contains pre-rendered MP4 bounce clips per event.

Naming format:

`bounce_{bounce_frame}_{shot_id}.mp4`

6. Existing Stream Simulator

File

`streaming-move-main/main.py`

Already supports:

- progressive HLS release
- sliding live window
- live playback simulation

This should remain untouched.

7. Existing React Player

Directory

`streaming-move-main/player/`

Already includes:

- React + Vite
 - hls.js integration
 - rolling segment buffer
 - review mode architecture
 - local blob VOD playback
 - DVR-style live window
-

What Needs To Be Built

Build a unified triple-camera review screen with:

- synchronized live playback
- instant camera switching
- Hotstar-style event markers
- event-based seeking
- review mode

- synchronized replay
- three-camera bounce clip inspection

Everything must work inside a single-page interface.

Core Architecture Requirement

IMPORTANT: All 3 streams must stay connected simultaneously

Do NOT reconnect streams during camera switching.

The frontend must:

- create 3 hls.js instances
- keep all 3 buffering continuously
- maintain rolling buffers for all cameras
- show only one video visually at a time

Camera switching should only:

1. map playback position using sync API
2. seek target video
3. swap visible player

Expected result:

- near-instant switching
 - no black screens
 - no rebuffers
-

Review Mode Requirement

When entering review mode:

- all live hls.js connections must be destroyed
- all downloads must stop immediately

- replay should use ONLY already-buffered segment bytes
- blob-based local VOD playlists should be generated for all 3 cameras

During review:

- seeking should work
- camera switching should work
- NO network requests should occur

When "Go Live" is clicked:

- blob playlists destroyed
 - memory cleaned
 - live HLS sessions restored
-

Backend Assignment

Create

`streaming-move-main/tri_stream_server.py`

The server should:

At Startup

Load playlists

Read all:

- segment names
- durations

for:

- source
 - sink
 - HQ
-

Build frame → playback lookup maps

Using:

`hls_segment_frame_index.csv`

Convert:

frame number

→ segment index

→ timestamp offset

Build camera sync mappings

Using:

`hls_sync_1645_triple.csv`

Generate:

camera A timestamp

→ equivalent camera B timestamp

Build event metadata

Using:

`flight_shots.csv`

Convert bounce frames into:

- playback positions
 - per-camera timestamps
 - event metadata
-

Streaming Requirements

Run:

- 3 independent circular live streams
- continuous looping playback

- rolling HLS windows

Suggested ports:

- 8081 → SOURCE
 - 8082 → SINK
 - 8083 → HQ
-

Required APIs

GET /cameras

Returns:

```
{  
  "source": "...",  
  "sink": "...",  
  "hq": "..."  
}
```

GET /sync

Example:

/sync?from_camera=hq&from_seg=45

Returns:

- matching segment/time positions for all cameras
-

GET /events

Returns:

- all shots

- playback timestamps
 - bounce coordinates
 - clip filenames
 - shot metadata
-

GET /status

Returns:

- currently streamed segment per camera
-

Frontend Assignment

New Component

CameraSelector.jsx

Requirements:

- SOURCE / SINK / HQ tabs
 - active state styling
 - per-camera color indicator
 - callback-based switching
-

Modify App.jsx

Must manage:

- 3 hls.js instances
- 3 video refs
- active camera state
- review mode lifecycle
- sync-based seeking
- rolling buffers

Modify **SeekBar.jsx**

Add:

- event markers
- current-event highlighting
- prev/next event buttons
- hover tooltips

Keyboard support:

- left/right arrows jump events
-

New Component

EventPanel.jsx

Slide-up panel containing:

- 3 synchronized bounce clip players
- SOURCE / SINK / HQ labels
- playback controls
- close button
- event metadata

Must handle missing clips gracefully.

UI Expectations

Single-window experience.

Expected layout:

HEADER
VIDEO PLAYER
SEEK BAR WITH EVENTS

EVENT PANEL

The experience should feel similar to:

- Hotstar sports replay
 - VAR review systems
 - Hawk-Eye style inspection tools
-

Deliverables

Please submit the following:

1. GitHub Repository

Share:

- complete source code
 - setup instructions
 - README
 - architecture notes
-

2. Demo Video (Mandatory)

Record a short demo showing:

- all 3 streams running
- instant camera switching
- event marker navigation
- replay mode
- review without downloads
- event panel with 3 bounce clips

The demo should clearly show:

- buffering behavior
- switching latency
- replay workflow

3. Setup Instructions

Include:

- installation steps
 - dependencies
 - commands to run backend/frontend
 - assumptions made
-

Timeline

Expected completion:

1–2 days

If additional time is required, communicate early.

Evaluation Criteria

We will evaluate based on:

- Architecture quality
 - Stability
 - Playback smoothness
 - Code cleanliness
 - React structure
 - Sync accuracy
 - Review-mode implementation
 - UX quality
 - Debuggability
 - Execution speed
-

Notes

- You are free to improve the UI/UX.
- Keep implementation practical and production-oriented.
- Avoid unnecessary abstractions.
- Focus on smooth playback and synchronization correctness.
- Low-latency camera switching is extremely important.

Once the assignment is completed, please share:

- GitHub repository
- Demo video
- Any implementation notes

Our team will review the submission and get back to you regarding the next steps.